

CNN Plays on Stanford Cars Dataset

TensorFlow 2.0



Introduction

Play with CNNs on a more serious classification dataset.

Data

A variant of the original Stanford Cars Dataset which was preprocessed to be a classification dataset. See details in <https://www.kaggle.com/sungtheillest/vehicledetected-stanford-cars-data-classes-folder>. Data was downloaded to x99-server at `/home/yoav/dev/other/tf2.0/data/stanford-car-dataset-by-classes-folder`.

A note on why this dataset is harder than it looks. The classes are fine-grained -- distinguishing one model year of a saloon from the next -- so the differences that matter are small and local, while the differences that do not matter, such as colour, pose and background, are large and global. That is the opposite of the situation a network pre-

trained on general object classes is tuned for, and it is the reason transfer learning with frozen weights disappoints here.

~8000 training samples.

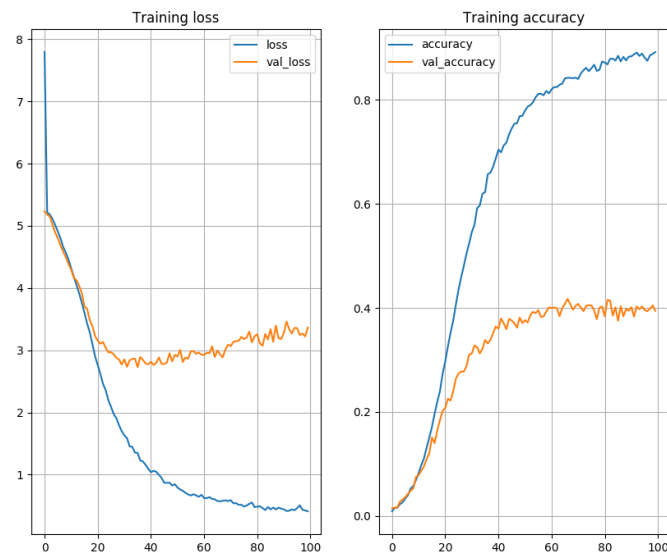
Training Exploration - VGG16

Doing transfer learning with the weights frozen (data is small and thus this seems like a reasonable thing to do). Transforming the input images through the VGG16 (frozen) backbone for speed. Transformed features are saved in `dev/other/tf2.0/data/stanford-car-dataset-by-classes-folder/vgg16_out`.

- Training with the following simple head:

```
x = Dense(512, activation='relu')(x)
x = Dropout(0.1)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.1)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.1)(x)
x = Dense(K, activation='softmax')(x)
```

Training accuracy is ~0.9 (goes to ~1.0 with no regularization); test accuracy ~0.4



There is a large difference between train and test accuracy. Since train accuracy is good (and in fact, with no regularization, the network overfits perfectly to the train set), I tried to increase regularization: Dropout with 0.2 is reducing both accuracies; Dropout with 0.4 is not converging at all

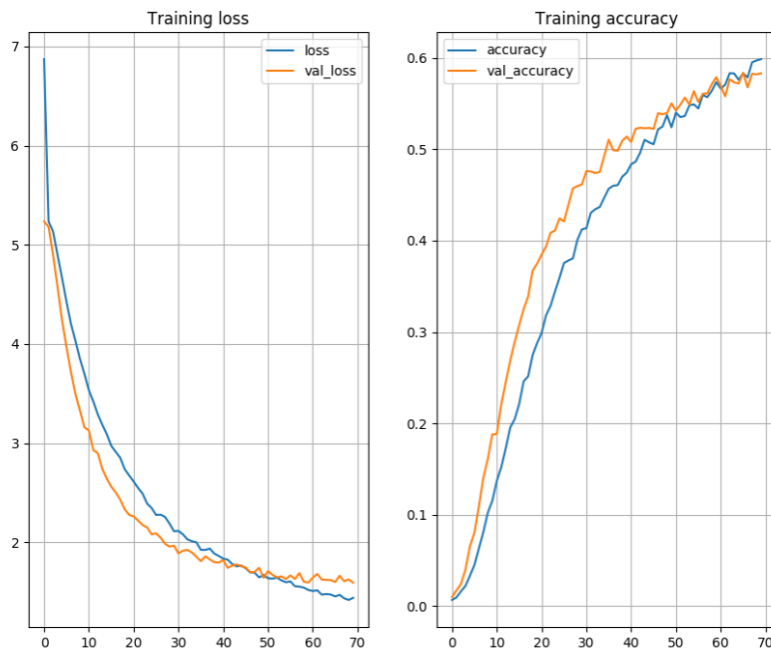
- The first dense layer is huge with more than 10M parameters. Trying to use 1x1 convolutions to reduce the features' depth dimension (the spatial dimensions are 7x7, which cannot be significantly reduced). No improvement.....

Most of that capacity is spent before the network has learned anything useful, which is why the 1x1 convolution below helps as much as it does.

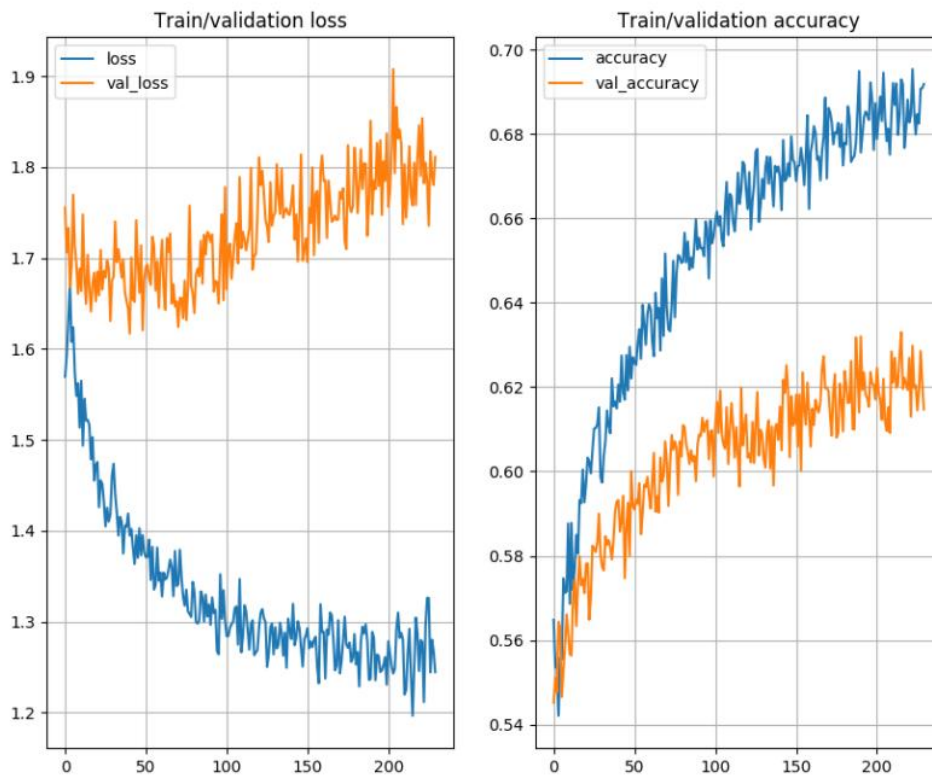
```
i = ptm.input
x = ptm.output
x = Conv2D(64, (1, 1), activation='relu')(x)
x = Flatten()(x)
x = Dense(512, activation='relu')(x)
x = Dropout(0.2)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.2)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.2)(x)
```

```
x = Dense(K, activation='softmax')(x)
```

- With dropout of 0.1, and with decaying learning rate (), got slightly better results: train accuracy: 0.9877 , val_accuracy: 0.4622
- Trained with data augmentation (not using the transformed features), is much much slower, but is improving the test accuracy dramatically. See plot below after 70 epochs:



After more training (total of 300 epochs) this is what we get:



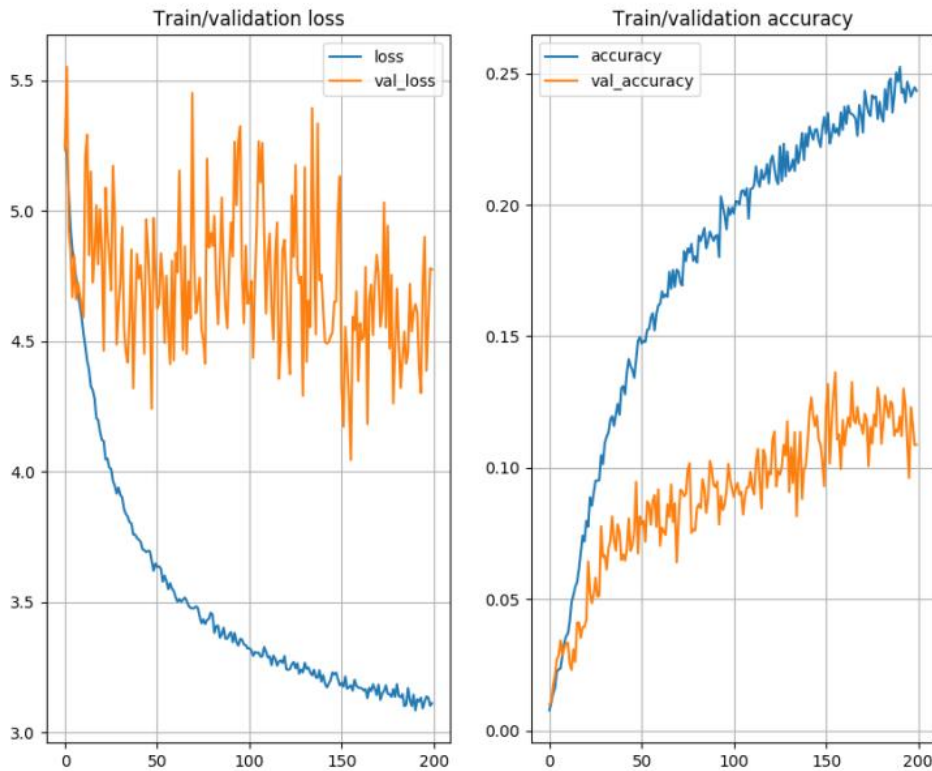
Training Exploration - Inception V3

Transformed features are saved in `dev/other/tf2.0/data/stanford-car-dataset-by-classes-folder/inception_v3_out`.

Inception V3 with the head above (with 1x1 conv of 64 channels) got ~ 45% test accuracy

With data augmentation the result is even worse -

Model InceptionV3



Training Exploration - ResNet50 V2

Transformed features are saved in `dev/other/tf2.0/data/stanford-car-dataset-by-classes-folder/resnetv2_out`.

ResNet50 V2 with the head above (with 1x1 conv of 64 channels) got ~ 35% test accuracy

On the larger train2 training set (50 epochs): accuracy: 0.781; val_accuracy: 0.369

Next Steps

Summary:

VGG16 based network with data augmentation produces the best results - ~0.6 test accuracy. Checkpoints for best model are in

~/dev/other/tf2.0/training_checkpoints/stanford-car-dataset/vgg16/1/

- Increase train set:
 - Increase train set by 50% on the expense of the test set.
 - New folders: train2, test2 with sizes 11400, 4700 respectively (before they were approximately equal)